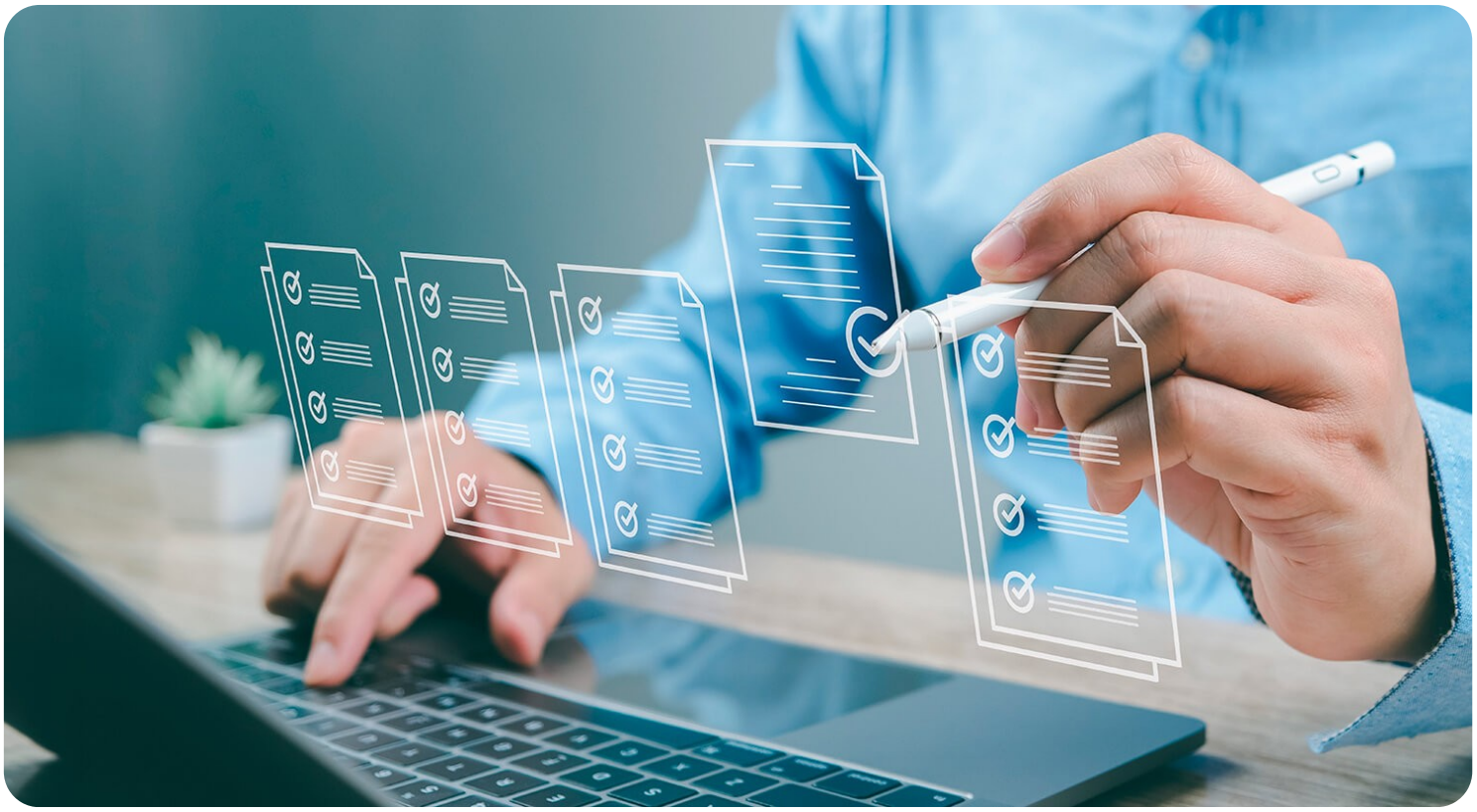


Semana 5

Desarrollo de Aplicaciones Móviles (DSY2204)

Formato de respuesta - SeñalSegura - Semana 5

Nombre estudiante:	Pablo Andrés Fuentes Castillo
Asignatura: Desarrollo de Aplicaciones Móviles	Carrera: Ingeniería en Desarrollo de Software
Profesor: Miguel Emilio Puebla Cuero	Fecha: 14 de septiembre de 2026

Descripción de la actividad

En esta quinta semana, se realizará la actividad sumativa 2, llamada "Integrando Kotlin a la aplicación móvil con Android Studio", donde deberán integrar a la app creada en el framework Android Studio, las funcionalidades básicas y avanzadas del lenguaje de programación Kotlin, con una entrega de encargo que contenga la documentación del desarrollo y código del proyecto comprimido en ZIP.

Instrucciones específicas

A continuación, se detallan las instrucciones y requisitos de la actividad sumativa 2, deberás leer cada uno de ellos y apuntar al cumplimiento.

Características del producto:

- Adaptar al lenguaje de programación Kotlin la aplicación móvil de accesibilidad para personas con discapacidad sensorial auditiva desarrollada en la experiencia de aprendizaje 1.
- La aplicación móvil deberá ser diseñada con componentes UI específicamente para personas con discapacidad sensorial auditiva, facilitando a los usuarios su comunicación (escribir y hablar) en su entorno cotidiano, mejorando su calidad de vida al transformar sus dispositivos móviles en una herramienta que los integra al mundo real y digital.

Requerimientos generales:

Para el desarrollo de la entrega solicitada considerar los siguientes aspectos:

- Implementación el lenguaje de programación Kotlin en cada componente UI de Material Design y Jetpack Compose integrado en entrega de la experiencia de

aprendizaje 1 (input, botones, tablas, grillas, vínculos, textos, combo box, check list y radio buttons). Adaptar las views de Login, Registro y Recuperar contraseña al lenguaje de programación Kotlin.

- Generar un array en Kotlin que almacene los datos de 5 usuarios y sus contraseñas, registrados previamente desde el formulario de la view de Registro.

Requerimientos técnicos

- Desarrollar la aplicación con el Framework Android Studio.
- Generar y compartir el repositorio Git del proyecto.
- Integrar el lenguaje de programación Kotlin al código fuente de la app móvil.

Entrega de encargo con documentación

Esta actividad tendrá dos partes y se entregará todo en **un solo ZIP.**
En la primera parte, una vez desarrollado el proyecto desde Android Studio, se deberá comprimir la carpeta del código del proyecto completo en formato .zip desde Git y adjuntarlo en espacio indicado en el AVA.
En la segunda parte, se deberá documentar todo el proceso de desarrollo del proyecto, describiendo el rendimiento de las views diseñadas, restricciones y el seguimiento de los 5 riesgos técnicos del proyecto definidos en la experiencia de aprendizaje 1, además de los accionables de mitigación de cada riesgo definido. Todo ello **en este mismo documento de formato de respuesta, a continuación de estas instrucciones** y subirlo en el ZIP que entregarás con el desarrollo del Proyecto y este documento del formato de respuesta debe subirse en formato PDF.

- Entregar el producto totalmente finalizado, según el avance solicitado en esta entrega.
- No se aceptarán entregas fuera de plazo ni por otro medio que no sea AVA.
- Todos los productos desarrollados con IA serán evaluados con nota mínima.

Restricciones en esta entrega

La aplicación móvil debe ser adaptativa a múltiples dispositivos Android que cuenten con una conexión activa a internet.

Formato del texto:

- Fuente: Arial.
- Tamaño de fuente: 12 puntos.
- Interlineado: 1,5 líneas.
- Alineación del texto: a la izquierda.

Iniciar documentación desde la siguiente página.

Datos del documento

Histórico de revisiones

Versión	Fecha	Descripción/cambio	Autor
0.1	08-09-2026	Análisis de pauta, continuidad del proyecto y riesgos.	Pablo Fuentes
0.2	12-09-2026	Integración de Kotlin avanzado, búsqueda y accesibilidad.	Pablo Fuentes
1.0	14-09-2026	Pruebas, documentación y preparación de la entrega final.	Pablo Fuentes

Información del proyecto

Organización	Duoc UC - Campus Virtual
Sección	Campus Virtual (sección no informada)
Proyecto (nombre)	SeñalSegura
Fecha de inicio	18 de agosto de 2026
Fecha de término	14 de septiembre de 2026
Patrocinador principal	Asignatura DSY2204
Docente	Miguel Emilio Puebla Cuero

Alcance del proyecto. Problema, solución propuesta y contexto del proyecto.
Problema. Las personas con discapacidad auditiva pueden enfrentar barreras cuando una conversación, una instrucción o una alerta depende exclusivamente del sonido. En situaciones cotidianas, laborales o de emergencia, esto puede retrasar la comprensión y limitar una comunicación autónoma. **Solución propuesta.** SeñalSegura es una aplicación Android que permite escribir y mostrar mensajes en tamaño grande, reproducirlos mediante voz sintetizada y usar alertas accesibles. Incorpora frases rápidas, filtros combinables, alertas visuales, vibración, alto contraste y descripciones semánticas para TalkBack. **Contexto.** Esta entrega continúa el prototipo aprobado en semanas anteriores y lo adapta a Kotlin avanzado con Jetpack Compose y Material Design. Usa información local para demostrar login, registro y recuperación, y aborda mensajes generales, de seguridad y salud sin reemplazar protocolos oficiales de emergencia. **Incluye.** Login, registro, recuperación, pantalla de comunicación, arreglo inicial de cinco usuarios, texto a voz, grilla adaptativa, cinco alertas, filtros por categoría, búsqueda, favoritas, control de errores y Kotlin avanzado. **Fuera de alcance.** Base de datos remota, autenticación real, envío de correos, reconocimiento de voz, interpretación automática de lengua de señas, geolocalización y publicación en Google Play.

Funciones esperadas del software a desarrollar.

Funciones esperadas

- Validar el acceso mediante un Array inicial de cinco usuarios y contraseñas locales.
- Registrar nuevos usuarios durante la ejecución con perfil y preferencia de comunicación.
- Simular la recuperación de contraseña verificando el correo ingresado.
- Mostrar mensajes escritos en un panel de gran tamaño y alto contraste.
- Reproducir mensajes mediante el motor de texto a voz del dispositivo.
- Seleccionar frases rápidas de las categorías General, Seguridad y Salud.
- Buscar y filtrar cinco alertas accesibles por texto, categoría y favoritas.
- Activar una alerta visual acompañada de respuesta háptica.
- Validar el mensaje mediante una función de extensión y retroalimentar errores sin cerrar la app.
- Adaptar las grillas a diferentes anchos y entregar resúmenes semánticos a TalkBack.

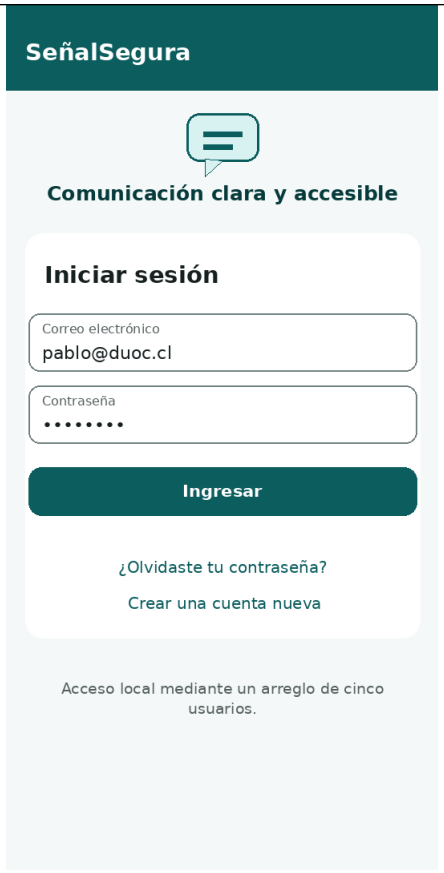


Figura 1. Inicio de sesión

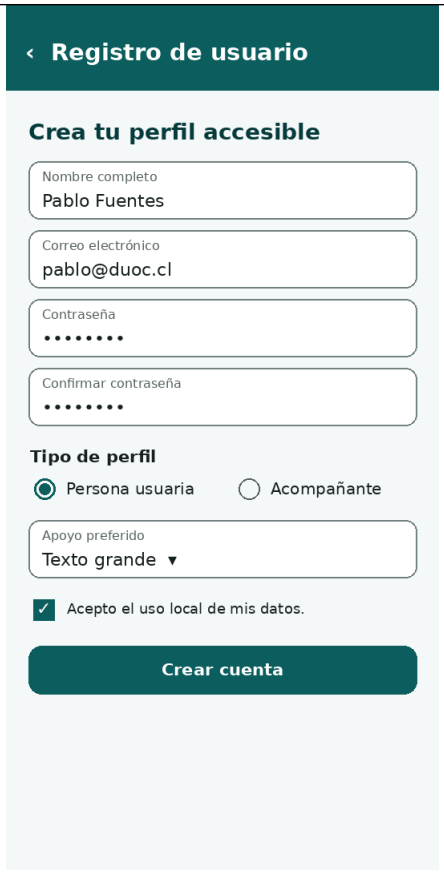


Figura 2. Registro



Figura 3. Recuperación



Figura 4. Comunicación

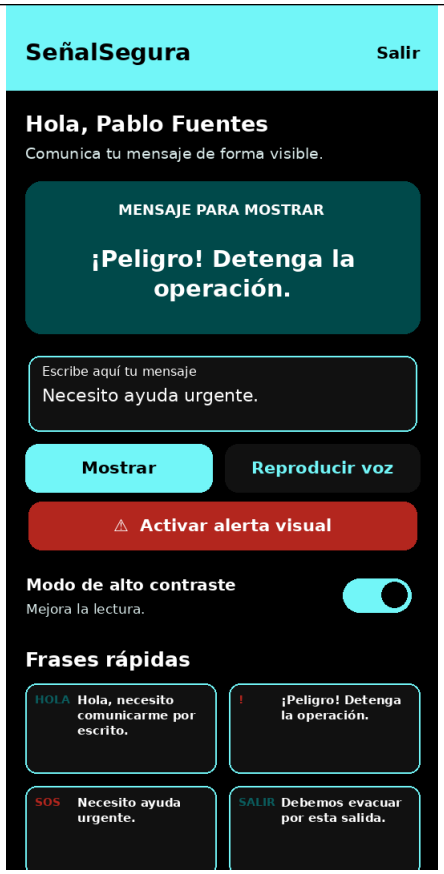


Figura 5. Alto contraste



Figura 6. Alerta visual

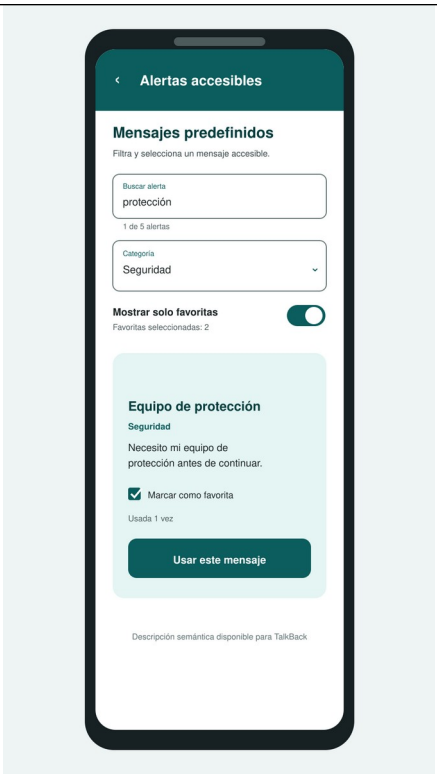


Figura 7. Alertas y búsqueda

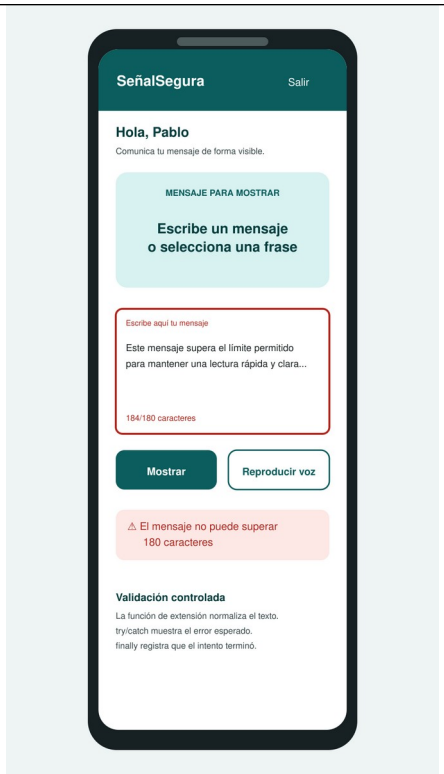


Figura 8. Validación del mensaje

Rendimiento esperado del software a desarrollar.
Criterio general. Las operaciones locales deben responder sin bloqueos visibles. La navegación, validación, filtrado y selección de mensajes tienen una meta inferior a un segundo en un dispositivo compatible. Las listas contienen solo cinco alertas y ocho frases, por lo que el filtrado en memoria es inmediato; la voz depende del motor TTS instalado. **View Login.** Presenta identidad visual, correo, contraseña, validación local y enlaces a registro y recuperación. Informa errores por texto y vibración. **View Registro.** Captura nombre, correo, contraseñas, perfil, preferencia y aceptación

mediante input, radio buttons, combo box y checklist. Impide correos duplicados.

View Recuperación. Comprueba el correo en los datos locales y muestra una respuesta visual clara. No envía un correo real en esta fase.

View Inicio. Permite escribir, ampliar y reproducir mensajes; filtrar una grilla de frases rápidas; activar alto contraste y cerrar sesión.

View Alertas accesibles. Muestra cinco objetos de alerta en una grilla adaptativa. Combina búsqueda, categoría y favoritas mediante una lambda; registra usos en un Map y entrega un resumen semántico a TalkBack.

View Alerta visual. Muestra una ventana de alto impacto con color, texto grande y vibración, evitando depender únicamente de señales sonoras.

Adaptabilidad. La grilla usa GridCells.Adaptive, por lo que aumenta o reduce el número de columnas según el ancho disponible. Los formularios permiten desplazamiento vertical y mantienen controles de al menos 48 dp de alto.

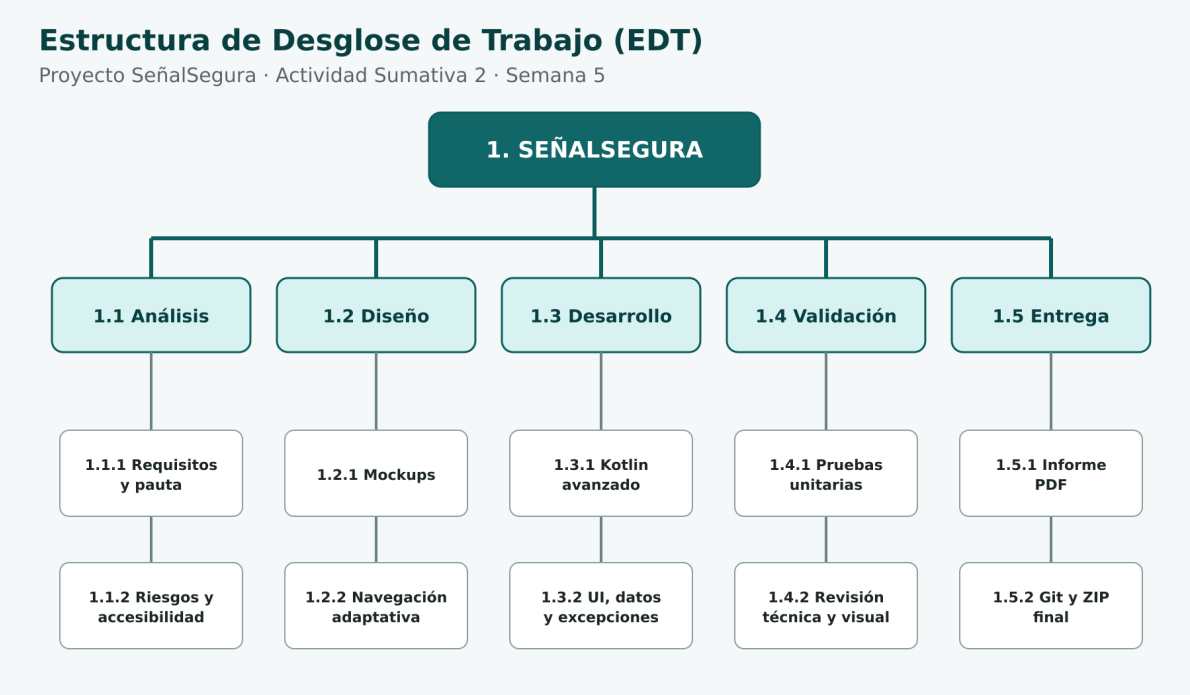
Validación y estabilidad. El mensaje se normaliza y limita a 180 caracteres. La excepción específica se captura para informar el problema en pantalla y el bloque finally registra que el intento terminó; las excepciones no relacionadas no se silencian.

Restricciones del proyecto.

- La aplicación se ejecuta únicamente en Android; no existe versión para iOS.
- Requiere Android 7.0 o superior (API 24) y una instalación de Android Studio compatible con el proyecto.
- Se requiere conexión activa a internet para sincronizar por primera vez Gradle y las dependencias; las funciones locales de comunicación pueden operar sin conexión después de esa sincronización.
- El texto a voz depende de que exista un motor TTS con idioma español instalado y habilitado.
- Los datos de registro se mantienen en memoria durante la ejecución y no persisten después de cerrar completamente la aplicación.
- La recuperación de contraseña es demostrativa: no envía mensajes ni modifica credenciales reales.
- La vibración, el audio y ciertas prestaciones pueden estar limitados en un emulador.
- El prototipo no sustituye alarmas certificadas, procedimientos de emergencia ni sistemas de seguridad industrial.
- El alcance académico no incluye servidor, base de datos, cifrado, analítica ni publicación en Google Play.
- La compilación depende de Gradle 9.5, Android Gradle Plugin 9.3.1, JDK 21 para Gradle y Compile SDK 37; se debe respetar esta combinación al abrir el proyecto.

Estructura de desglose de trabajo. Definición de Diagrama y diccionario EDT. Adjuntar diagrama y diccionario.

La EDT divide la actividad sumativa 2 en cinco paquetes: análisis, diseño, desarrollo, validación y entrega. El desarrollo distingue Kotlin avanzado de la integración de UI, datos y excepciones; la validación separa pruebas unitarias de revisión técnica y visual. El diagrama y el diccionario se repiten en anexos.



Resumen de riesgos

Listado de riesgos relacionado al desarrollo de S.W. Indicar riesgo, etapa o fase en que se presenta, la probabilidad de que ocurra, magnitud o impacto de este riesgo por etapa en el proceso.

Probabilidad: alta, media, baja.

Impacto: alto, significativo, moderado, inferior y bajo.

Riesgo	Fase	Probabilidad	Impacto	Acción de mitigación
Incompatibilidad entre SDK, AGP y Gradle	Configuración	Media	Alto	Estado: controlado. Mitigación: fijar Gradle 9.5, AGP 9.3.1, JDK 21 y SDK 37; sincronizar antes de programar.
El emulador no inicia por virtualización o gráficos	Pruebas	Media	Significativo	Estado: controlado. Mitigación: API 35, arranque en frío y gráficos por software si reaparece.
Motor de texto a voz no disponible en español	Ejecución	Media	Moderado	Estado: residual. Mitigación: verificar TTS y conservar el texto grande como alternativa.
Contraste o tamaño insuficiente para el usuario	Diseño UI	Baja	Alto	Estado: mitigado. Mitigación: alto contraste, texto grande, grillas adaptativas y TalkBack.
Pérdida de usuarios registrados al cerrar la app	Datos	Alta	Moderado	Estado: aceptado. Mitigación: informar persistencia en memoria y planificar Room.

Definición de artefactos.

Listar y describir los artefactos o entregables que serán administrados y entregados durante el desarrollo del proyecto.

Artefacto	Descripción
Código fuente Android	Proyecto Kotlin con Jetpack Compose, Material Design, navegación y configuración Gradle.
Repositorio Git	Repositorio local en rama main con commits de Semanas 2, 3, 4 y 5, README y evidencias técnicas.
Mockups	Ocho representaciones de vistas y estados, incluidas búsqueda de alertas y validación del mensaje.
EDT y diccionario	Desglose jerárquico del trabajo y descripción de sus paquetes.
Pruebas unitarias	Veintiocho casos para usuarios, frases, alertas, ViewModels, extensiones y manejo de errores.
Informe técnico	Documento DOCX de trabajo y PDF final elaborados sobre el formato oficial de Semana 5.

Condiciones de aceptación para cierre del proyecto
Condiciones que se deben cumplir para dar termino al proyecto y margen de tolerancia de aceptación de defectos.

- El proyecto abre y sincroniza en Android Studio utilizando el JDK 21 configurado para Gradle.
- El login valida al menos cinco usuarios locales y rechaza credenciales incorrectas.
- Las vistas de login, registro, recuperación e inicio permiten navegación coherente hacia adelante y atrás.
- El registro valida campos, contraseñas, aceptación y duplicidad de correo.
- La pantalla principal muestra texto grande, reproduce voz, filtra frases y activa una alerta visual.

- La pantalla de alertas contiene cinco objetos y combina búsqueda, categoría y favoritas.
- La solución integra una función de orden superior que retorna una lambda, una función inline y lambdas de colección y UI.
- Existen funciones y propiedades de extensión aplicadas al mensaje, conteo de palabras y resumen para TalkBack.
- Los mensajes superiores a 180 caracteres generan un error visible y controlado mediante try/catch/finally.
- El modo de alto contraste mantiene legibles los textos y controles principales.
- La grilla se adapta a diferentes anchos y los formularios pueden desplazarse sin recortar contenido.
- Las 28 pruebas unitarias incluidas deben finalizar sin fallas en un entorno con las dependencias sincronizadas.
- La entrega incluye código, repositorio Git local, README e informe técnico en un único ZIP.
- No se aceptan fallas que impidan iniciar sesión o navegar; se toleran diferencias menores de apariencia entre versiones de Android.

Anexos

Anexo 1: Diagrama EDT

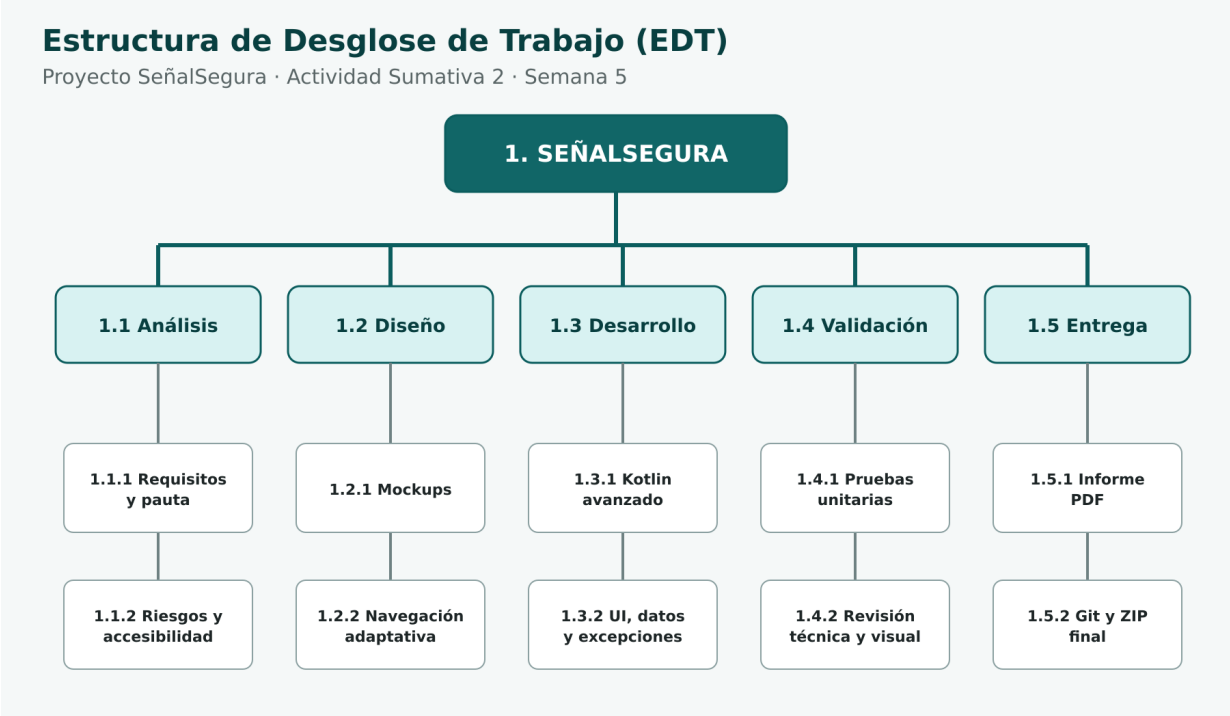


Figura 9. Estructura de Desglose de Trabajo de la actividad sumativa 2.

Anexo 2. Diccionario EDT

1.1 Análisis: Revisar la pauta, continuidad del proyecto, alcance y criterios de aceptación.

1.1.1 Requisitos y pauta: Trazar UI, Git, experiencia, Kotlin avanzado y documentación con los cinco criterios evaluados.

1.1.2 Riesgos y accesibilidad: Actualizar los cinco riesgos e identificar medidas para comunicación visual, voz, contraste y TalkBack.

1.2 Diseño: Actualizar la solución visual y el flujo de navegación de SeñalSegura.

1.2.1 Mockups: Representar ocho vistas o estados, incluidos filtro de alertas y validación del mensaje.

1.2.2 Navegación adaptativa: Mantener NavController, retorno coherente, scroll y grillas que responden al ancho disponible.

1.3 Desarrollo: Programar Kotlin avanzado e integrarlo con UI, modelos, datos y ViewModels.

1.3.1 Kotlin avanzado: Implementar funciones de orden superior, inline, lambdas, extensiones y try/catch/finally.

1.3.2 UI, datos y excepciones: Conectar filtros, Array de cinco usuarios, alertas, retroalimentación y excepción específica.

1.4 Validación: Comprobar reglas, navegación, accesibilidad y comportamiento esperado.

1.4.1 Pruebas unitarias: Mantener 28 pruebas sobre repositorios, ViewModels, extensiones y gestión de errores.

1.4.2 Revisión técnica y visual: Revisar fuentes, estructura, componentes, mockups y renderizado del informe.

1.5 Entrega: Preparar el producto final según el formato institucional.

1.5.1 Informe PDF: Completar el documento oficial y verificar visualmente todas sus páginas.

1.5.2 Git y ZIP final: Registrar el avance y comprimir proyecto, historial, evidencias e informe en un solo ZIP.

Anexo 3. Evidencia técnica de Kotlin avanzado

Los fragmentos siguientes identifican la implementación funcional; el código completo y sus pruebas se encuentran en el proyecto incluido en el ZIP.

Función de orden superior que retorna una lambda.

```
fun buildAlertPredicate(...): (AccessibleAlert) -> Boolean = { alert ->
    matchesCategory && matchesFavorite && matchesQuery
}
```

La lambda combina búsqueda, categoría y favoritas y luego se entrega a filter(predicate).

Función inline con lambdas como parámetros.

```
inline fun <T> executeAccessibleAction(
    action: () -> T, onError: (InvalidAccessibleMessageException) -> Unit,
    onFinally: () -> Unit
): T?
```

HomeViewModel la usa para actualizar el mensaje, mostrar el error esperado y registrar el cierre del intento.

Funciones y propiedades de extensión.

```
val String.wordCount: Int
fun String.toAccessibleMessage(maxLength: Int = 180): String
val AccessibleAlert.accessibilitySummary: String
```

Estas extensiones cuentan palabras, normalizan/validan el texto y crean una descripción completa para TalkBack.

Gestión de errores.

```
try { action() }
catch (error: InvalidAccessibleMessageException) { onError(error) }
finally { onFinally() }
```

Se captura solo la excepción recuperable; errores ajenos a la validación continúan propagándose.

Pruebas y trazabilidad.

AdvancedKotlinTest · AlertRepositoryTest · AlertsViewModelTest · HomeViewModelTest

El conjunto contiene 28 pruebas. La evidencia detallada está en EVIDENCIA_KOTLIN_AVANZADO_SEMANA5.md.



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC
La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.